

L09 · Γραφήματα IV

Dijkstra & Ελάχιστα Συνδετικά Δέντρα

Algorithms Class Hub

Αλγόριθμοι και Πολυπλοκότητα (K17) · ΕΚΠΑ

Συντομότερες διαδρομές με βάρη (Dijkstra), ελάχιστα συνδετικά δέντρα (MST), η ιδιότητα αποκοπής και η ιδιότητα κύκλου, και οι άπληστοι αλγόριθμοι Prim και Kruskal.

1 Τι θα δούμε

Δύο μεγάλα προβλήματα σε ζυγισμένα γραφήματα, και τρεις **άπληστους** αλγορίθμους που τα λύνουν:

1. **Συντομότερη διαδρομή με θετικά βάρη** → **Dijkstra**, $O(m \log n)$.
2. **Ελάχιστο Συνδετικό Δέντρο (ΕΕΔ / MST)** → **Prim**, **Kruskal** (και η «αντίστροφη διαγραφή»), $O(m \log n)$.

Και οι τρεις είναι **άπληστοι**: σε κάθε βήμα παίρνουν την τοπικά καλύτερη απόφαση. Το ότι αυτό δίνει **παγκόσμια** βέλτιστη λύση δεν είναι προφανές — θα δούμε την απόδειξη μέσω δύο όμορφων ιδιοτήτων: της **ιδιότητας αποκοπής** και της **ιδιότητας κύκλου**.

Κλειδί

Σπουδαία διάλεξη για την εξεταστική. Σχεδόν κάθε χρονιά υπάρχει θέμα που ζητάει «τρέξε Dijkstra στο γράφημα...», «βρες το ΕΕΔ...», ή «εφάρμοσε την ιδιότητα αποκοπής για να αποδείξεις...». Επίσης είναι τα τελευταία θέματα όπου το BFS/DFS από προηγούμενα μαθήματα παίζει ως υπορουτίνα.

2 Συντομότερη Διαδρομή με Βάρη

2.1 Διατύπωση

Είσοδος: κατευθυνόμενο γράφημα $G = (V, A)$. Σε κάθε ακμή $e \in A$ ανατίθεται **θετικός** πραγματικός αριθμός $\ell_e > 0$ (το «μήκος» της).

Κόστος διαδρομής: το άθροισμα των μηκών των ακμών της.

Πρόβλημα: δοθέντων αφετηρίας s και προορισμού t , βρες την κατευθυνόμενη διαδρομή από s σε t με το **ελάχιστο** συνολικό μήκος.

2.2 Γιατί δεν αρκεί το BFS

Από το προηγούμενο μάθημα ξέρουμε ότι το BFS λύνει το πρόβλημα **χωρίς** βάρη. Σε ζυγισμένο γράφημα όμως, η διαδρομή με τις λιγότερες ακμές **δεν** είναι κατ' ανάγκη η μικρότερη σε κόστος: μια διαδρομή 3 ακμών μήκους 5 η καθεμία (σύνολο 15) είναι **καλύτερη** από μια διαδρομή 1 ακμής μήκους 100.

3 Αλγόριθμος Dijkstra

3.1 Ιδέα — άπληστη επέκταση από την s

Κρατάμε ένα σύνολο S από κορυφές **εξερευνημένες**, για τις οποίες έχουμε ήδη υπολογίσει την απόσταση από την s . Σε κάθε βήμα:

1. Για κάθε **μη εξερευνημένη** κορυφή v , υπολογίζουμε

$$\pi(v) = \min_{u \in S} \{d(u) + \ell_{u,v}\}$$

δηλαδή «αν φτάσεις στην v από κάποια $u \in S$, ποιο είναι το ελάχιστο κόστος;»

2. Διάλεξε την κορυφή v με το **ελάχιστο** $\pi(v)$.
3. Θέσε $d(v) := \pi(v)$, πρόσθεσε την v στο S .

Αρχικοποίηση: $S = \{s\}$, $d(s) = 0$.

Διαίσθηση

Διαισθητικά: φαντάσου ότι τοποθετείς ένα μαθηματικό «κύμα» στην s που διαδίδεται με ταχύτητα 1. Σε κάθε χρονική στιγμή, η επόμενη κορυφή που το κύμα φτάνει είναι ακριβώς η μη-εξερευνημένη με το μικρότερο $\pi(v)$. Είναι μια **γενίκευση του BFS** όπου το «επόμενο επίπεδο» δεν είναι μια ακμή μακριά, αλλά **όσο επιτρέπει η μικρότερη ακμή**.

3.2 Ψευδοκώδικας

```
Dijkstra(G, l, s):
  Για κάθε  $v \in V$ :
     $d[v] \leftarrow \infty$ 
     $\pi[v] \leftarrow \emptyset$ 
   $d[s] \leftarrow 0$ 
  PQ <- ουρά προτεραιότητας με όλες τις κορυφές (κλειδί = d)
  S <-  $\emptyset$ 
  Ενόσω PQ  $\neq \emptyset$ :
     $u \leftarrow \text{extractMin}(PQ)$ 
    S <- S  $\cup \{u\}$ 
    Για κάθε ακμή  $(u, v) \in A$ :
      Εάν  $v \notin S$  και  $d[u] + l(u, v) < d[v]$ :
         $d[v] \leftarrow d[u] + l(u, v)$ 
         $\pi[v] \leftarrow u$ 
        decreaseKey(PQ, v, d[v])
  επίστρεψε d,  $\pi$ 
```

Όταν τελειώσει, $d[v]$ είναι η συντομότερη απόσταση από την s στην v . Με τους «γονείς» $\pi[\cdot]$ μπορούμε να ανακατασκευάσουμε την ίδια τη διαδρομή — ακολουθώντας πίσω από t προς s .

3.3 Ορθότητα — αναλλοίωτη με επαγωγή στο $|S|$

Κλειδί

Αναλλοίωτη: για κάθε $u \in S$, $d(u)$ ισούται με το μήκος της **πραγματικής** συντομότερης διαδρομής $s \rightarrow u$.

Βάση: $|S| = 1$, $S = \{s\}$, $d(s) = 0$. ✓

Επαγωγικό βήμα: υποθέτουμε ότι η αναλλοίωτη ισχύει για $|S| = k$. Έστω v η κορυφή που μπαίνει στο S στο βήμα $k+1$ — αυτή που ελαχιστοποιεί το $\pi(\cdot)$ — και έστω (u, v) η ακμή που δίνει αυτό το ελάχιστο, δηλαδή $\pi(v) = d(u) + \ell_{u,v}$.

Θα δείξουμε ότι **κάθε** άλλη διαδρομή $P : s \rightarrow v$ έχει μήκος $\geq \pi(v)$. Έστω λοιπόν P μια τέτοια διαδρομή. Έστω (x, y) η **πρώτη** ακμή της P που «βγαίνει» από το S (δηλαδή $x \in S$, $y \notin S$). Έστω P' η υποδιαδρομή $s \rightarrow x$:

$$\begin{aligned} \ell(P) &\geq \ell(P') + \ell_{x,y} && \text{(αρχικό κομμάτι της } P) \\ &\geq d(x) + \ell_{x,y} && \text{(αναλλοίωτη επαγωγής για } x \in S) \\ &\geq \pi(y) && \text{(από τον ορισμό του } \pi(y)) \\ &\geq \pi(v) && \text{(ο Dijkstra διάλεξε } v \text{ ως minimum)} \end{aligned}$$

Άρα $\ell(P) \geq \pi(v) = d(v)$. ■

Προσοχή

Σημαντικός περιορισμός: η αναλλοίωτη χρησιμοποιεί ότι **όλα τα βάρη είναι θετικά**. Αν υπήρχε αρνητική ακμή, θα μπορούσε να «μειώσει» το κόστος μιας υποδιαδρομής που βγαίνει και ξαναμπαίνει στο S — άκυρη απόδειξη, και πράγματι ο Dijkstra **δεν** δουλεύει με αρνητικά βάρη. Σε εκείνη την περίπτωση χρειάζεται **Bellman-Ford** ($O(nm)$).

3.4 Πολυπλοκότητα

- **Χωρίς δομή δεδομένων:** σε κάθε βήμα ψάχνουμε γραμμικά την κορυφή με το μικρότερο π . $O(n)$ ανά extraction, n extractions $\rightarrow O(n^2)$. Καλό για πυκνά γραφήματα.
- **Με δυαδικό σωρό (heap):** ένας σωρός υποστηρίζει extractMin σε $O(\log n)$ και decreaseKey σε $O(\log n)$. Συνολικά: n extractMin $\rightarrow O(n \log n)$ · μέχρι m decreaseKey $\rightarrow O(m \log n)$.
- **Σύνολο:** $O((n + m) \log n) = O(m \log n)$ (για συνεκτικά γραφήματα όπου $m \geq n - 1$).

Με Fibonacci heap πέφτει σε $O(m + n \log n)$ — αλλά εκείνο πέρα από το μάθημα.

4 Ελάχιστο Συνδεδετικό Δέντρο (ΕΕΔ / MST)

4.1 Διατύπωση

Είσοδος: μη κατευθυνόμενο **συνεκτικό** γράφημα $G = (V, E)$ με κόστη c_e σε κάθε ακμή.

Επικαλύπτον δέντρο (spanning tree): υποσύνολο $T \subseteq E$ τέτοιο ώστε (1) το T είναι **δέντρο** (συνεκτικό + ακυκλικό) και (2) επικαλύπτει **όλες** τις κορυφές V . Από προηγούμενο μάθημα ξέρουμε ότι κάθε δέντρο με n κορυφές έχει $n - 1$ ακμές.

Πρόβλημα: βρες ένα επικαλύπτον δέντρο **ελάχιστου κόστους**, δηλαδή ελαχιστοποίησε $\sum_{e \in T} c_e$.

4.2 Γιατί δεν αρκεί η ωμή βία

Κλειδί

Θεώρημα του Cayley. Το πλήρες γράφημα K_n έχει n^{n-2} διαφορετικά επικαλύπτοντα δέντρα.

Άρα η εξαντλητική απαρίθμηση είναι **ασύμφορη** — π.χ. το K_{10} έχει 100 εκατομμύρια. Χρειαζόμαστε έξυπνο αλγόριθμο.

Πραγματικές εφαρμογές: σχεδίαση δικτύων (τηλεφωνικά, ηλεκτρικά, υδρευτικά, καλωδιακά, οδικά), προσεγγιστικοί αλγόριθμοι για NP-δύσκολα προβλήματα (TSP, δέντρο Steiner), max-bottleneck paths, LDPC κώδικες για επιδιόρθωση λαθών, καταχώρηση εικόνας με εντροπία Rényi, το spanning tree protocol σε δίκτυα Ethernet (αποφυγή loops), και ανάλυση συστάδων (clustering).

5 Τρεις Άπληστοι Αλγόριθμοι

Αλγόριθμος	Στρατηγική	Πολυπλοκότητα
Kruskal	ακμές σε αύξουσα σειρά κόστους, προσθέτεις αν δεν δημιουργεί κύκλο	$O(m \log n)$
Αντίστροφη διαγραφή	ακμές σε φθίνουσα σειρά κόστους, αφαιρείς αν δεν αποσυνδέει το γράφημα	$O(m \log n)$
Prim	ξεκίνα από κορυφή s , σε κάθε βήμα πρόσθεσε τη φθηνότερη ακμή που «μεγαλώνει» το δέντρο	$O(m \log n)$

Όλοι δίνουν την ίδια λύση. Διαφορετική διατύπωση, ίδιο αποτέλεσμα.

6 Κύκλοι, Αποκοπές, και η Τομή τους

Πριν αποδείξουμε ότι οι παραπάνω αλγόριθμοι είναι σωστοί, χρειαζόμαστε δύο ορισμούς.

6.1 Αποκοπή

Αποκοπή (cut): οποιοδήποτε υποσύνολο κορυφών $A \subset V$. Το αντίστοιχο **σύνολο ακμών αποκοπής** D είναι το σύνολο των ακμών που έχουν **το ένα άκρο στο** A και το άλλο στο $V \setminus A$.

Διαίσθηση

Διαισθητικά: η αποκοπή «κόβει» το γράφημα σε δύο πλευρές. Οι ακμές αποκοπής είναι αυτές που «περνάνε» τη γραμμή.

6.2 Το βασικό λήμμα

Κλειδί

Λήμμα (κύκλος-αποκοπή). Ένας κύκλος C και ένα σύνολο ακμών αποκοπής D τέμνονται σε **άρτιο** πλήθος ακμών.

Απόδειξη: αν ο κύκλος είναι ολόκληρος μέσα στο A ή ολόκληρος έξω, η τομή είναι κενή ($= 0$). Αλλιώς, κάθε φορά που ο κύκλος «μπαίνει» στο A (μέσω ακμής του D), πρέπει επίσης να «βγει» (επίσης μέσω ακμής του D). Άρα οι ακμές του C στο D ζευγαρώνουν $\rightarrow$ άρτιο πλήθος. ■

6.3 Ιδιότητα Αποκοπής

Κλειδί

Ιδιότητα Αποκοπής. Έστω $S \subset V$ οποιαδήποτε αποκοπή και έστω e η ακμή **ελάχιστου κόστους** στο σύνολο ακμών αποκοπής $D(S)$. Τότε **το ΕΕΔ περιέχει την e .**

Απόδειξη (με ανταλλαγή). Έστω T^* το ΕΕΔ, και υποθέτουμε για αντίφαση ότι $e \notin T^*$.

1. Προσθέτοντας την e στο T^* δημιουργείται μοναδικός κύκλος C (δέντρο + ακμή = ένας κύκλος).
2. Η e ανήκει στον C **και** στο $D(S)$.
3. Από το λήμμα κύκλου-αποκοπής, υπάρχει **τουλάχιστον μία άλλη** ακμή $f \in C \cap D(S)$, $f \neq e$.
4. Φτιάχνουμε νέο δέντρο: $T' = T^* \cup \{e\} \setminus \{f\}$. Είναι επικαλύπτων δέντρο (αφαίρεση μιας ακμής κύκλου διατηρεί τη συνεκτικότητα).
5. Αφού $c_e < c_f$ (η e είναι η ελάχιστη στο D), έχουμε $\text{cost}(T') < \text{cost}(T^*)$ — **αντίφαση** ότι το T^* ήταν ελάχιστο. ■

Εφαρμογή: Prim και Kruskal εξασφαλίζουν ότι κάθε ακμή που προσθέτουν είναι ελάχιστη σε κάποια αποκοπή $\rightarrow$ από την ιδιότητα ανήκει στο ΕΕΔ.

6.4 Ιδιότητα Κύκλου

Κλειδί

Ιδιότητα Κύκλου. Έστω C οποιοσδήποτε κύκλος και έστω f η ακμή **μέγιστου κόστους** στο C . Τότε **το ΕΕΔ δεν περιέχει την f .**

Απόδειξη (συμμετρική). Υποθέτουμε για αντίφαση ότι $f \in T^*$.

1. Αφαιρώντας την f από το T^* δημιουργείται αποκοπή S (δέντρο – ακμή = δύο συνιστώσες).
2. Η f ανήκει στον C **και** στο $D(S)$.
3. Από το λήμμα, υπάρχει $e \in C \cap D(S)$, $e \neq f$.
4. $T' = T^* \cup \{e\} \setminus \{f\}$ είναι επικαλύπτων δέντρο.
5. Αφού $c_e < c_f$ (η f είναι η μέγιστη στο C), $\text{cost}(T') < \text{cost}(T^*)$. **Αντίφαση.** ■

Εφαρμογή: αντίστροφη διαγραφή και Kruskal εξασφαλίζουν ότι κάθε ακμή που **αποκλείουν** είναι μέγιστη σε κάποιο κύκλο.

Διαίσθηση

Παραδοχή απλοποίησης: στις αποδείξεις υποθέσαμε ότι **όλα τα κόστη είναι μοναδικά** (κανένα ζεύγος ακμών δεν έχει ίδιο κόστος). Όταν υπάρχουν ισοτιμίες, οι αλγόριθμοι παίρνουν αυθαίρετες αποφάσεις και υπάρχουν πολλά «ελάχιστα» δέντρα. Όλα έχουν το ίδιο **συνολικό κόστος** — αλλά διαφορετικές συγκεκριμένες ακμές.

7 Αλγόριθμος του Prim

Ιδέα. Ξεκινάμε με μία κορυφή και «μεγαλώνουμε» το δέντρο **προς τα έξω**. Σε κάθε βήμα, η αποκοπή είναι «κορυφές μέσα στο δέντρο» vs «κορυφές έξω». Από την **ιδιότητα αποκοπής**, η φθηνότερη ακμή που περνάει αυτή τη γραμμή ανήκει στο ΕΕΔ — οπότε την παίρνουμε.

```
Prim(G, c, s):  
  Για κάθε  $v \in V$ :  
     $\alpha[v] \leftarrow \infty$  // κόστος επισύναψης της  $v$  στο  $S$   
     $\pi[v] \leftarrow \emptyset$   
   $\alpha[s] \leftarrow 0$   
   $Q \leftarrow$  ουρά προτεραιότητας με όλες τις κορυφές (κλειδί =  $\alpha$ )  
   $S \leftarrow \emptyset$   
   $T \leftarrow \emptyset$   
  Ενόσω  $Q \neq \emptyset$ :  
     $u \leftarrow \text{extractMin}(Q)$   
     $S \leftarrow S \cup \{u\}$   
    Εάν  $\pi[u] \neq \emptyset$ :  
       $T \leftarrow T \cup \{(u, \pi[u])\}$   
    Για κάθε ακμή  $e = \{u, v\}$  του  $G$ :  
      Εάν  $v \notin S$  και  $c_e < \alpha[v]$ :  
         $\alpha[v] \leftarrow c_e$   
         $\pi[v] \leftarrow u$   
         $\text{decreaseKey}(Q, v, \alpha[v])$   
  επίστρεψε  $T$ 
```

Παρατήρηση: μοιάζει εξαιρετικά με Dijkstra. Η διαφορά είναι ότι το **κλειδί** $\alpha[v]$ δεν είναι «κόστος διαδρομής από s » αλλά «κόστος της ελάχιστης ακμής που συνδέει τη v με το δέντρο». Άρα ο Prim είναι **τοπικός** (κοιτάει μόνο μία ακμή), ενώ ο Dijkstra είναι **συσσωρευτικός** (αθροίζει την πορεία).

Πολυπλοκότητα. Ίδια ανάλυση με Dijkstra: $O(m \log n)$ με δυαδικό σωρό, $O(n^2)$ με απλό πίνακα.

8 Αλγόριθμος του Kruskal

Ιδέα. Ταξινομούμε όλες τις ακμές σε **αύξουσα** σειρά κόστους. Σαρώνουμε με τη σειρά:

- **Περίπτωση 1:** αν η ακμή $e = \{u, v\}$ δημιουργεί κύκλο στο τρέχον $T \rightarrow$ απορρίπτουμε (ιδιότητα κύκλου: η e είναι η μεγαλύτερη ή ίση στον κύκλο, αφού όλες οι ήδη μπασμένες είναι φθηνότερες).
- **Περίπτωση 2:** αλλιώς $\rightarrow$ προσθέτουμε στο T (ιδιότητα αποκοπής: αν δεν δημιουργεί κύκλο, ενώνει δύο συνεκτικές συνιστώσες, και η e είναι η ελάχιστη στην αποκοπή «μία συνιστώσα vs την άλλη»).

Σταματάμε όταν το T έχει $n - 1$ ακμές.

```

Kruskal(G, c):
  Ταξινόμησε τις ακμές: c_1 <= c_2 <= ... <= c_m
  T <- ∅
  Για κάθε u ∈ V:
    MakeSet(u)                // κάθε κορυφή ξεκινά σε δικό της σύνολο
  Για i = 1 έως m:
    {u, v} = e_i
    Εάν Find(u) ≠ Find(v):    // όχι στην ίδια συνιστώσα -> όχι κύκλος
      T <- T ∪ {e_i}
      Union(u, v)
  επίστρεψε T

```

Ο έλεγχος «οι u, v είναι ήδη συνδεδεμένες;» γίνεται με τη δομή **Union-Find** που θα δούμε αναλυτικά σε επόμενο μάθημα (δομές δεδομένων).

Πολυπλοκότητα:

- Ταξινόμηση ακμών: $O(m \log m) = O(m \log n)$.
- m κλήσεις σε Find/Union: $O(m \cdot \alpha(n))$ με union-by-rank + path compression, όπου α είναι η **αντίστροφη Ackermann** — πρακτικά σταθερά.
- **Σύνολο:** $O(m \log n)$ (η ταξινόμηση είναι το κυρίαρχο κόστος).

9 Σύγκριση Prim vs Kruskal

	Prim	Kruskal
Δομή που μεγαλώνει	ένα δέντρο που μεγαλώνει από κορυφή	δάσος από πολλά δέντρα που συγχωνεύονται
Δομή δεδομένων	priority queue πάνω σε κορυφές	sort + Union-Find
Πολυπλοκότητα	$O(m \log n)$	$O(m \log n)$
Καταλληλότερος για	πυκνά γραφήματα	αραιά γραφήματα, παράλληλη υλοποίηση

Στην εξεταστική: συχνά ζητείται να εφαρμοστούν και οι δύο σε ένα συγκεκριμένο γράφημα. Δίνουν την ίδια λύση (αν τα κόστη είναι μοναδικά), αλλά **η σειρά** που προσθέτουν τις ακμές διαφέρει.

Σχόλιο: Αντίστροφη Διαγραφή. Λιγότερο γνωστή αλλά κομψή παραλλαγή: ξεκίνα με **όλες** τις ακμές, σάρωσε σε **φθίνουσα** σειρά κόστους, και **διέγραψε** κάθε ακμή της οποίας η αφαίρεση δεν αποσυνδέει το γράφημα. Λειτουργεί λόγω της ιδιότητας κύκλου. $O(m \log n)$ επίσης.

10 Ιστορία των αλγορίθμων ΕΕΔ

Αλγόριθμος	Πολυπλοκότητα
Borůvka (1926)	$O(m \log n)$
Jarník (1930) / Prim (1957) / Dijkstra (1959)	$O(m \log n)$
Kruskal (1956)	$O(m \log n)$
Yao (1975) / Cheriton-Tarjan (1976)	$O(m \log \log n)$
Fredman-Tarjan (1987)	$O(m + n \log n)$ (Fibonacci heap)
Chazelle (2000)	$O(m \alpha(m, n))$
Karger-Klein-Tarjan (1995)	$O(m)$ τυχαίοποιημένος
;	$O(m)$ ντετερμινιστικά — ανοιχτό

Η ντετερμινιστική γραμμική πολυπλοκότητα παραμένει **ανοιχτό πρόβλημα**. Μερικά από τα σημαντικότερα ονόματα της επιστήμης των υπολογιστών έχουν δουλέψει σε αυτό.

Ανακεφαλαίωση

Τι κρατάμε από αυτή τη διάλεξη

1. **Dijkstra** — απλός αλγόριθμος για συντομότερες διαδρομές με **θετικά** βάρη. Επεκτείνει την «εξερευνημένη ζώνη» κάθε φορά κατά την πιο κοντινή κορυφή. $O(m \log n)$ με σωρό.
2. **ΕΕΔ** — βρες δέντρο με $n - 1$ ακμές που συνδέει όλες τις κορυφές και έχει ελάχιστο συνολικό κόστος.
3. **Ιδιότητα αποκοπής** — η ελάχιστη ακμή σε κάθε αποκοπή ανήκει στο ΕΕΔ.
4. **Ιδιότητα κύκλου** — η μέγιστη ακμή σε κάθε κύκλο δεν ανήκει στο ΕΕΔ.
5. **Prim** — μεγαλώνει ένα δέντρο από μία κορυφή, διαλέγει σε κάθε βήμα τη φθηνότερη ακμή που το επεκτείνει. $O(m \log n)$.
6. **Kruskal** — σαρώνει ακμές σε αύξουσα σειρά κόστους, προσθέτει αν δεν δημιουργεί κύκλο. Χρησιμοποιεί Union-Find. $O(m \log n)$.
7. **Στη χείριστη ανάλυση** οι τρεις αλγόριθμοι είναι ίδιοι. Στην εξεταστική, η εφαρμογή σε συγκεκριμένο γράφημα είναι ένα πολύ συχνό θέμα.

Σημειώσεις από το Algorithms Class Hub — μελετητικός οδηγός για το μάθημα «Αλγόριθμοι και Πολυπλοκότητα (Κ17)», ΕΚΠΑ.